Introduction rapide à l’utilisation du logiciel R

Jean Paul Maalouf
linkedin.com/in/jean-paul-maalouf
www.statistique-formation.fr

Plan

Plan – Introduction rapide à R

1. Introduction
2. Installation
3. Démarrer un projet R
4. R : Syntaxe
    1. Un langage orienté-objet
    2. Fonctions et Packages
    3. Principaux types de données et commentaires
    4. Syntaxe classique et Tidyverse

Plan – Introduction rapide à R

5. Les tableaux de données
   1. Importer un jeu de données en data frame
   2. Caractériser un data frame
   3. Accéder à différents éléments d'un data frame suivant les numéros des lignes/colonnes
   4. Sélectionner des colonnes
   5. Sélectionner des lignes (filtrer)
   6. Trier un data frame
   7. Créer de nouvelles colonnes
   8. Effectuer automatiquement un calcul sur chaque colonne
   9. Effectuer des calculs par groupe
   10. Grouper les opérations : utilisation du *pipe*
6. Graphiques avec R

1. Introduction

R défini en quelques points

- Logiciel de manipulation de données et de statistique.
- Libre et Open Source (donc **gratuit**)
- Toute personne peut contribuer à son développement.
- Outil multiplateforme (PC, Mac, Linux).
- Permet de programmer.
- Très grande communauté.
- Des dizaines de milliers de fonctionnalités.

Comment R fonctionne-t-il ?

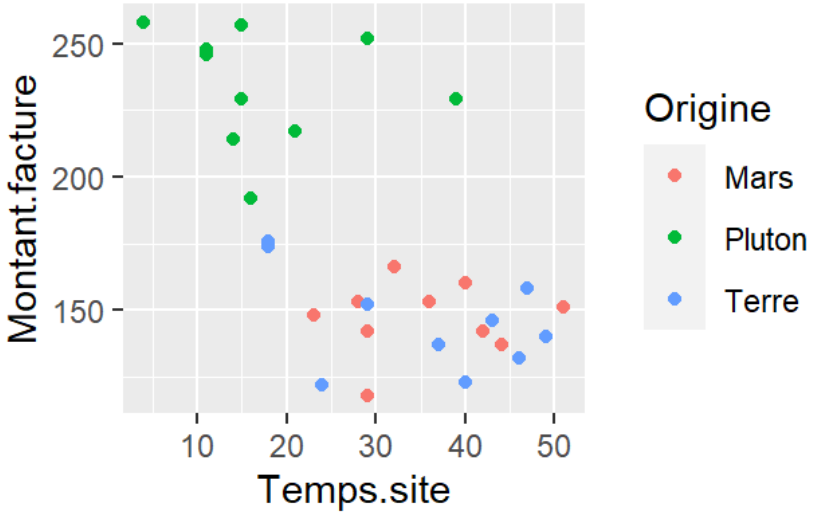
- R fonctionne en **lignes de code**. Chaque ligne exécute une opération.
- **Script** : ensemble de lignes de code. Peut être commenté, sauvegardé et réutilisé → automatiser l'analyse !
- Exemple sur jeu de données : clients d'une plateforme de vente de chaussures en ligne

	A	B	C	D	E	F	G	H	I	J	K
1	**Client**	**Origine**	**Sexe**	**Taille**	**Pointure**	**Poids**	**Temps passé sur le site**	**Marque préférée**	**Montant Facture**		
2	Willard	Mars	M	9	26	50	28	Marque A	153		
3	Derrick	Pluton	M	102	40	335	39	Marque D	229		
4	Olivia	Mars	F	13	31	68	29	Marque A	118		
5	Marc	Mars	M	20	29	81	40	Marque C	160		
6	Alice	Pluton	F	116	33	378	11	Marque B	248		
7	Elijah	Mars	M	22	30	88	44	Marque C	137		
8	Clyde	Mars	M	27	29	105	42	Marque C	142		
9	Angel	Mars	F	31	28	116	51	Marque A	151		
10	Randy	Terre	M	4	29	36	18	Marque A	174		
11	Robin	Terre	M	5	33	40	40	Marque C	123		

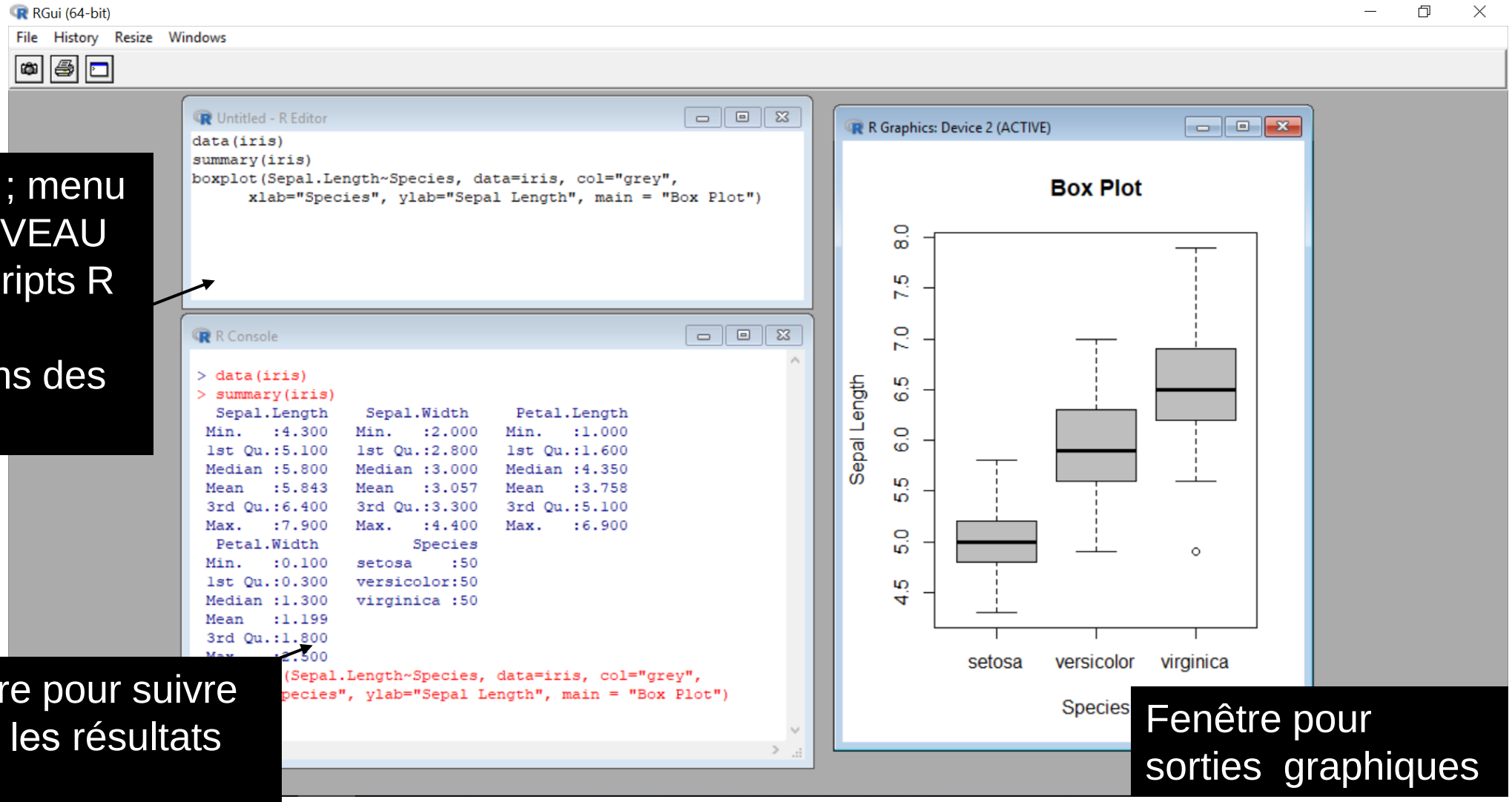
Manipuler R : exemple de script et résultats. chaussures est un tableau de données

```
# Script
names(chaussures)

mean(chaussures$Pointure)

qplot(x = Temps.site, y = Montant.facture,
      color = Origine, data = chaussures)
```

```
# Résultat
> names(chaussures)
"Client" "Origine" "Sexe"
"Marque.pref"  "Taille" "Pointure"
"Poids" "Temps.site"
"Montant.facture"
> mean(chaussures$Pointure)
[1] 29.63333
```



Que voit-on à l'ouverture du logiciel ?



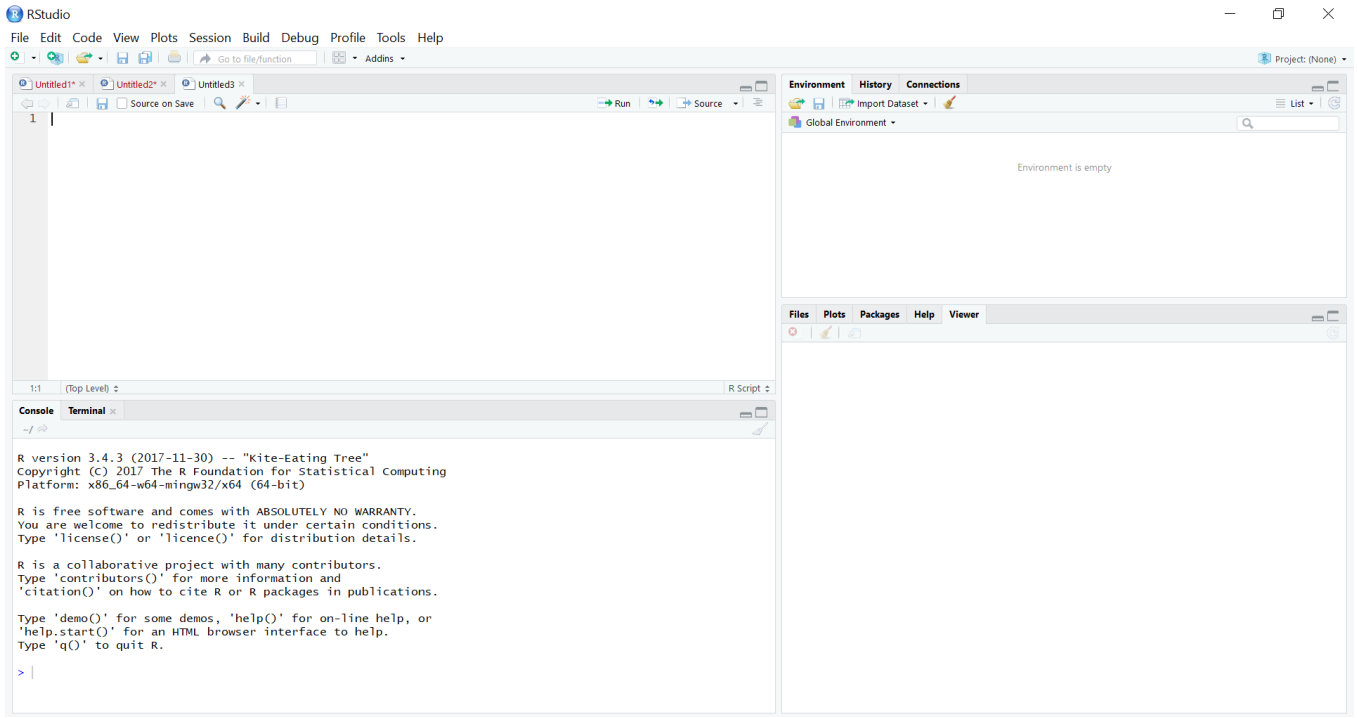
Editeur de script ; menu FICHIER > NOUVEAU SCRIPT . Les scripts R peuvent être sauvegardés dans des fichiers .R

Console = Fenêtre pour suivre l'exécution et voir les résultats texte

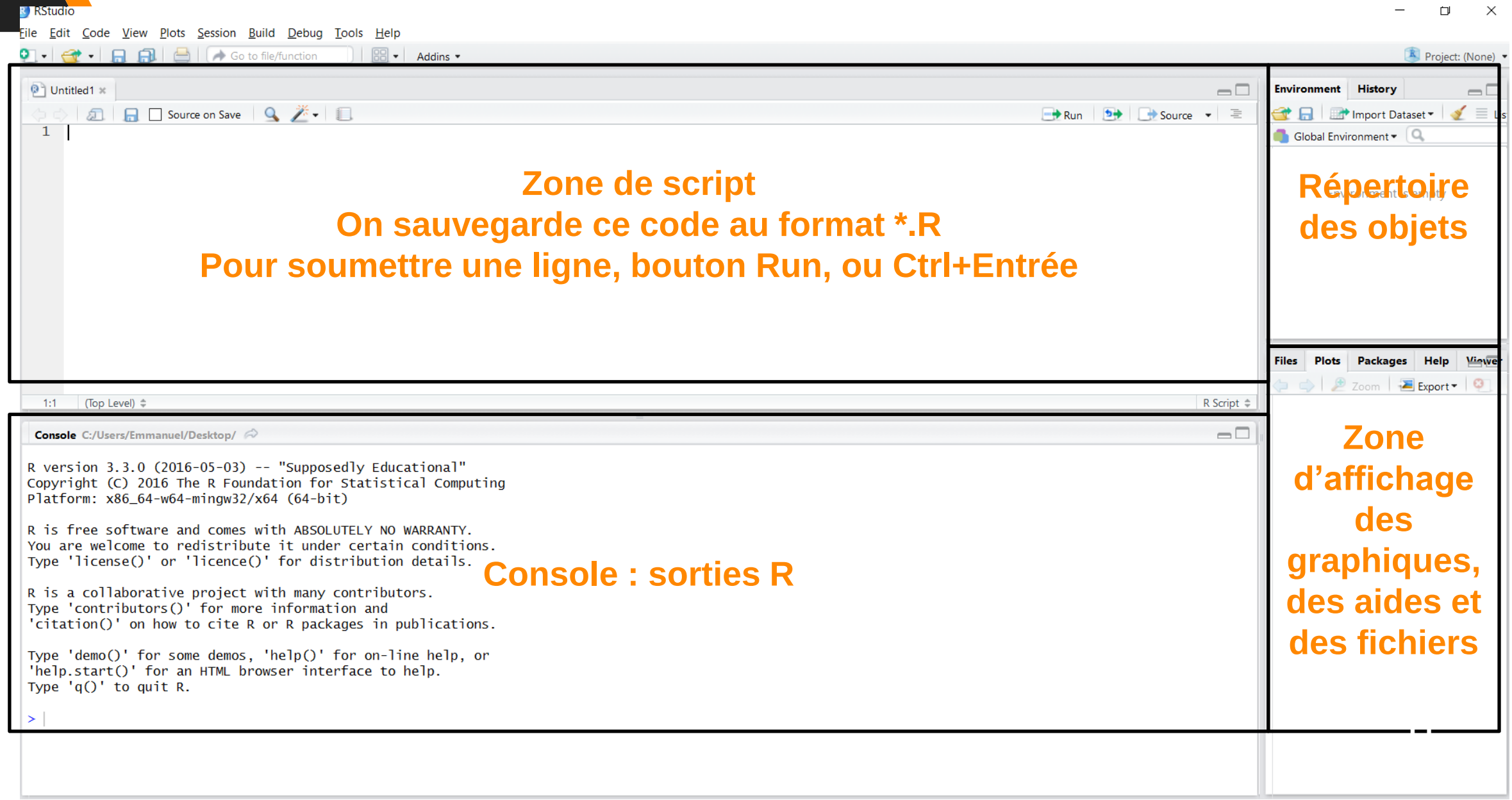
Fenêtre pour sorties graphiques

RStudio : présentation

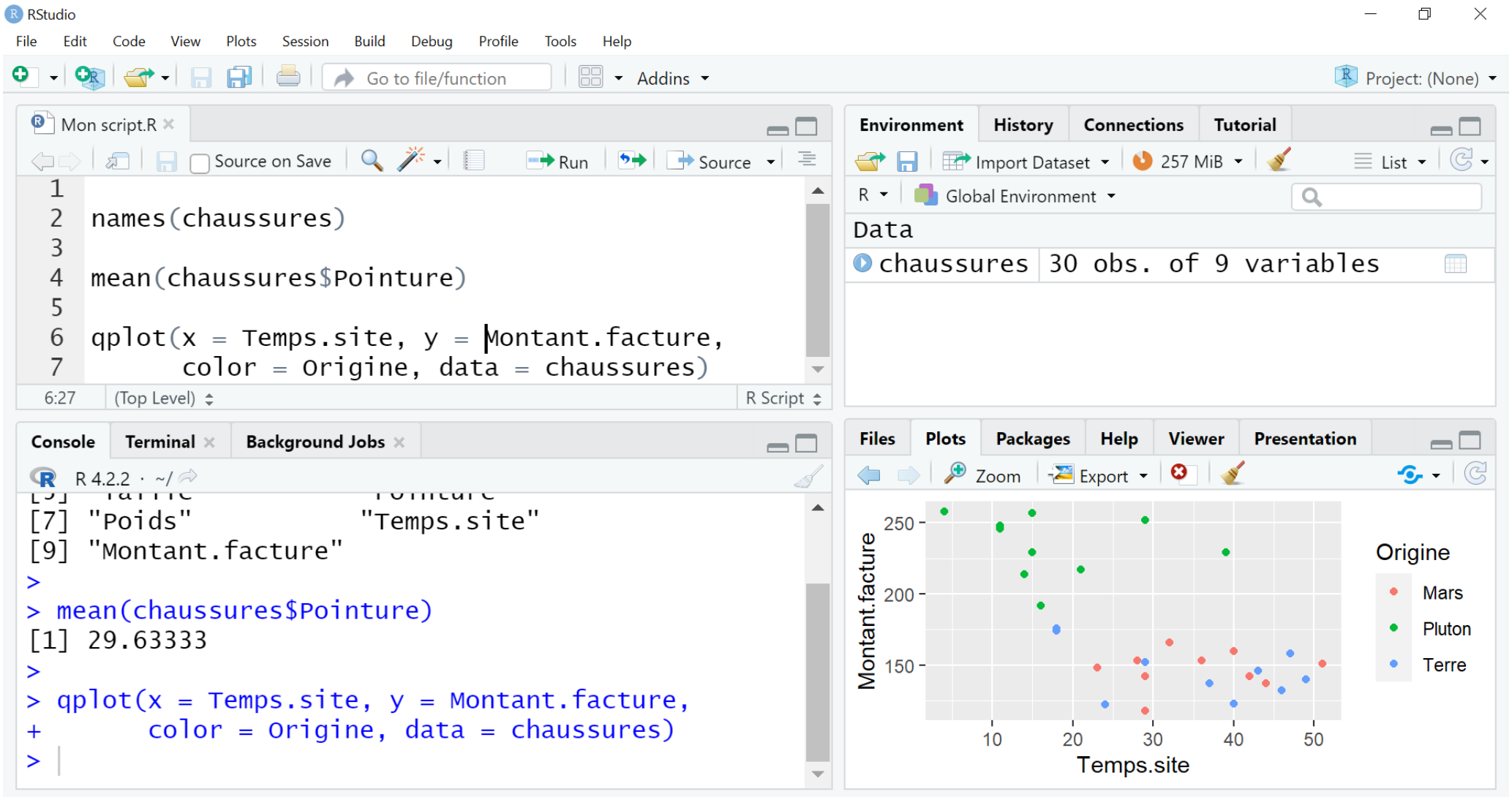
- Rstudio est une interface qui vous permet de coder et gérer les projets r de manière plus efficace et conviviale.



R doit être installé au préalable!!


RStudio
File Edit Code View Plots Session Build Debug Tools Help
Go to file/function
Addins
Project: (None)
Untitled1
Source on Save
Run
Source
Zone de script
On sauvegarde ce code au format *.R
Pour soumettre une ligne, bouton Run, ou Ctrl+Entrée
1:1
(Top Level)
R Script
Environment
History
Import Dataset
Global Environment
Répertoire des objets
Files
Plots
Packages
Help
Viewer
Zoom
Export
Zone d'affichage des graphiques, des aides et des fichiers
Console C:/Users/Emmanuel/Desktop/
R version 3.3.0 (2016-05-03) -- "Supposedly Educational"
Copyright (C) 2016 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)
R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.
R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.
Console : sorties R

L'exemple précédent vu sous RStudio



2. Installation

Etape 1 : Installer R

- R peut être téléchargé sur le site du CRAN (the Comprehensive R Archive Network).
- Pour le mettre à jour, il est necessaire d'effectuer une nouvelle installation.

CRAN
Mirrors
What's new?
Search
CRAN Team

About R
R Homepage
The R Journal

Software
R Sources
R Binaries
Packages
Task Views
Other

Documentation
Manuals
FAQs
Contributed

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- Download R for Linux (Debian, Fedora/Redhat, Ubuntu)
- Download R for macOS
- Download R for Windows

Choisir la plateforme qui convient

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2022-10-31, Innocent and Trusting) R-4.2.2.tar.gz, read what's new in the latest version.
- Sources of R alpha and beta releases (daily snapshots, created only in time periods before a planned release).
- Daily snapshots of current patched and development versions are available here. Please read about new features and bug fixes before filing corresponding feature requests or bug reports.
- Source code of older versions of R is available here.
- Contributed extension packages

Etape 2 : Installer RStudio

- R doit être installé au préalable !
- Taper RStudio sur un moteur de recherche et naviguer sur le site ou cliquer direct ici : https://posit.co/download/rstudio-desktop/
- Télécharger l'exécutable qui convient (il y en a plusieurs)

OS	Download
Windows 10/11	RSTUDIO-2022.07.2-576.EXE
macOS 10.15+	RSTUDIO-2022.07.2-576.DMG
Ubuntu 18+/Debian 10+	RSTUDIO-2022.07.2-576-AMD64.DEB

…

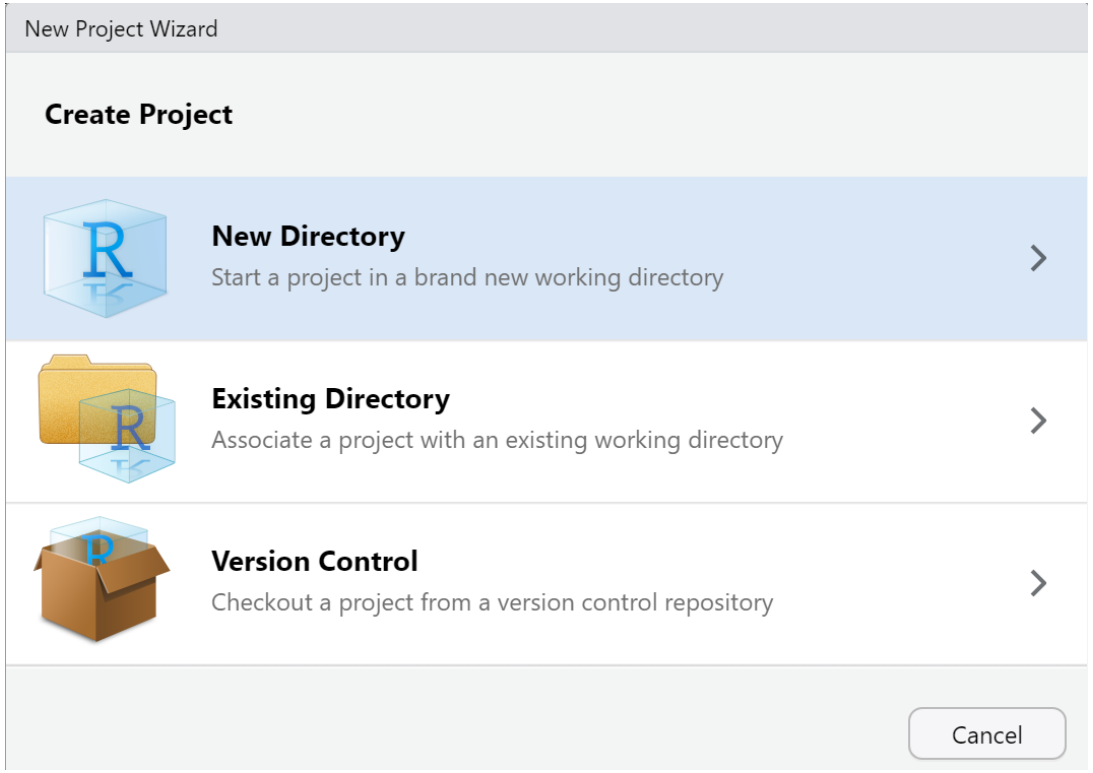
3. Démarrer un projet R

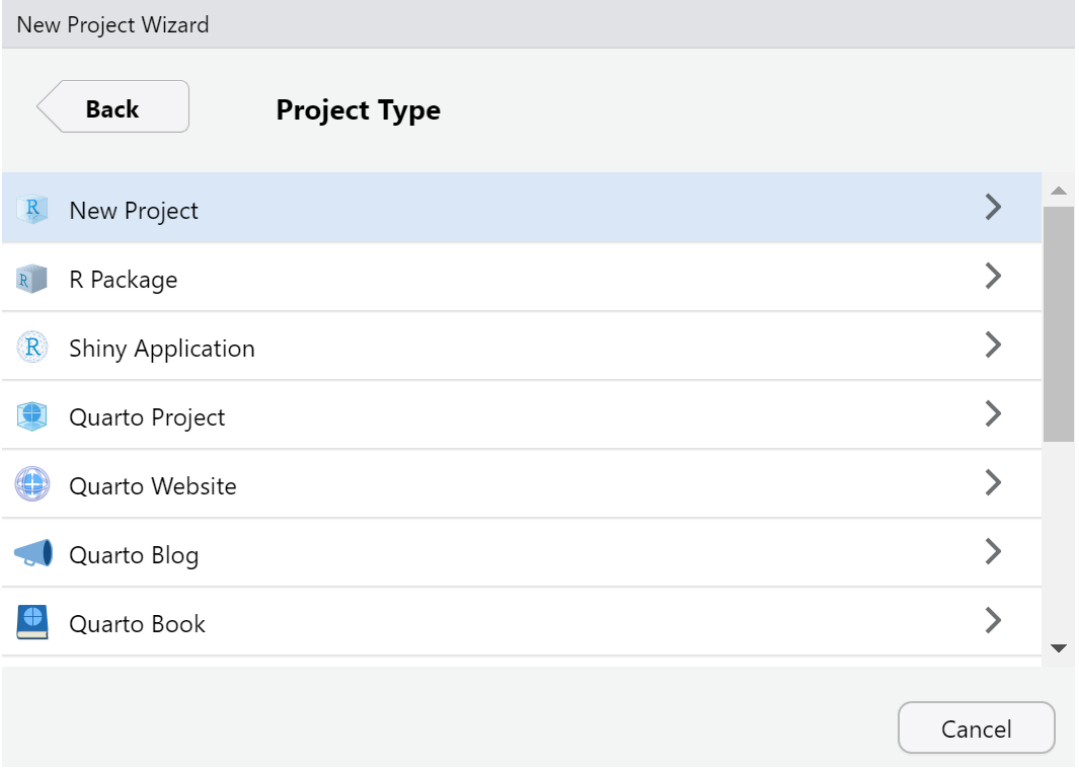
Démarrer un projet R

- Une bonne manière de travailler sur un projet R est de stocker tous les fichiers relatifs au projet dans le même dossier (répertoire courant).
- RStudio facilite la tâche de mise en place du projet.
- Pour démarrer un nouveau projet R, aller dans File/New Project

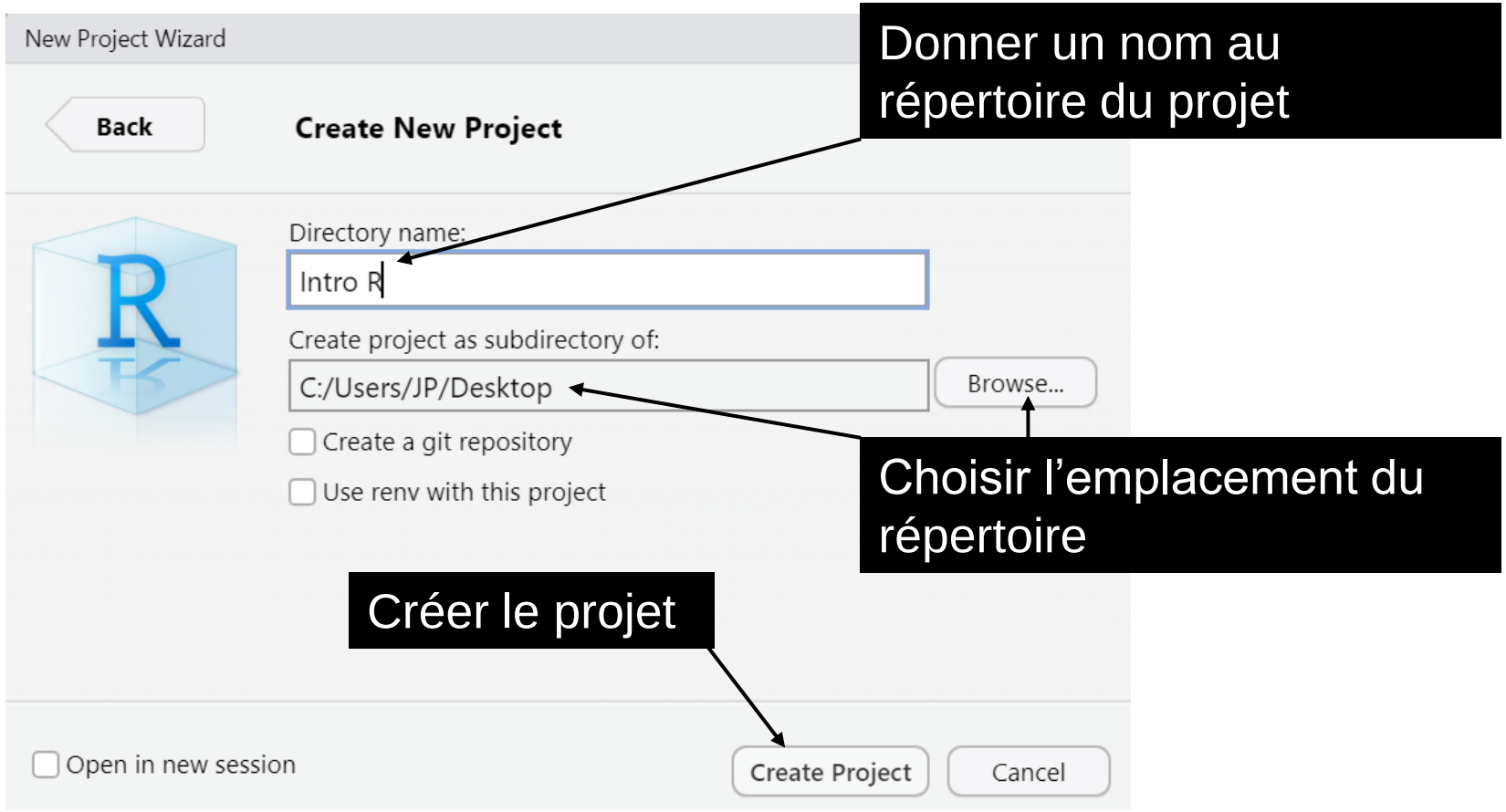
Démarrer un projet R

- Cliquer sur New Directory puis New Project



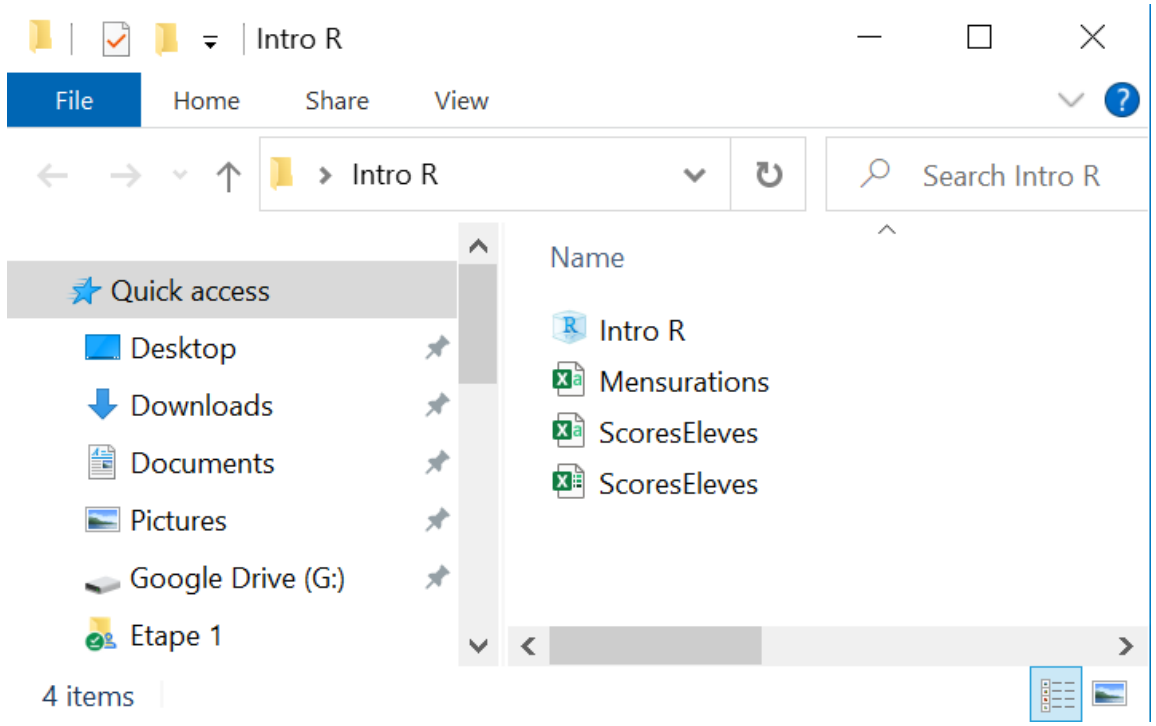


Démarrer un projet R



Démarrer un projet R

- Le dossier est créé avec le fichier Projet Formation.Rproj à l'intérieur
- Le projet pourra être relancé à tout moment depuis ce fichier (Rstudio s'ouvrira automatiquement).
- Nous y glissons les jeux de données sur lesquels nous souhaitons travailler. Exemple : ScoresEleves.xlsx, ScoresEleves.csv et Mensurations.csv



Démarrer un projet R

Intro R - RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Go to file/function | Addins | Intro R

Console | Terminal | Background Jobs

R 4.2.2 · C:/Users/JP/Desktop/Intro R/

```
R version 4.2.2 (2022-10-31 ucrt) -- "Innocent and Trusting"
Copyright (C) 2022 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |
```

Environment | History | Connections | Tutorial

Import Dataset | 140 MiB | List

R | Global Environment

Environment is empty

Files | Plots | Packages | Help | Viewer | Presentation

New Folder | New Blank File | Delete | Rename | More

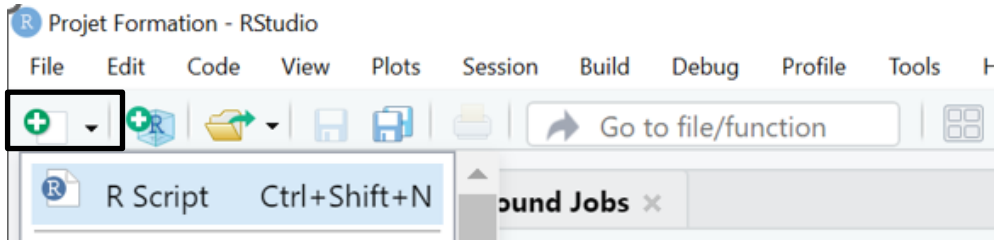
C: > Users > JP > Desktop > Intro R

Name	Size	Modified
..		
Intro R.Rproj	218 B	Nov 17, 2022, 9:43 PM
Mensurations.csv	33.8 KB	Nov 17, 2022, 9:23 PM
ScoresEleves.csv	1 KB	Nov 16, 2022, 5:27 PM
ScoresEleves.xlsx	11.2 KB	Nov 15, 2022, 3:00 PM

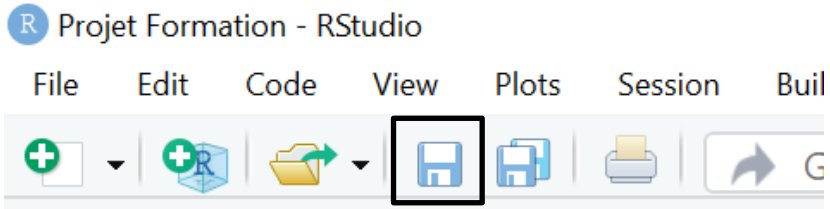
Le contenu du dossier apparaît ici dans RStudio

Prochaine étape : ouvrir un fichier de script

- Pour commencer à écrire des lignes de code, ouvrir un fichier de script. Cliquer sur la croix verte puis sur R Script

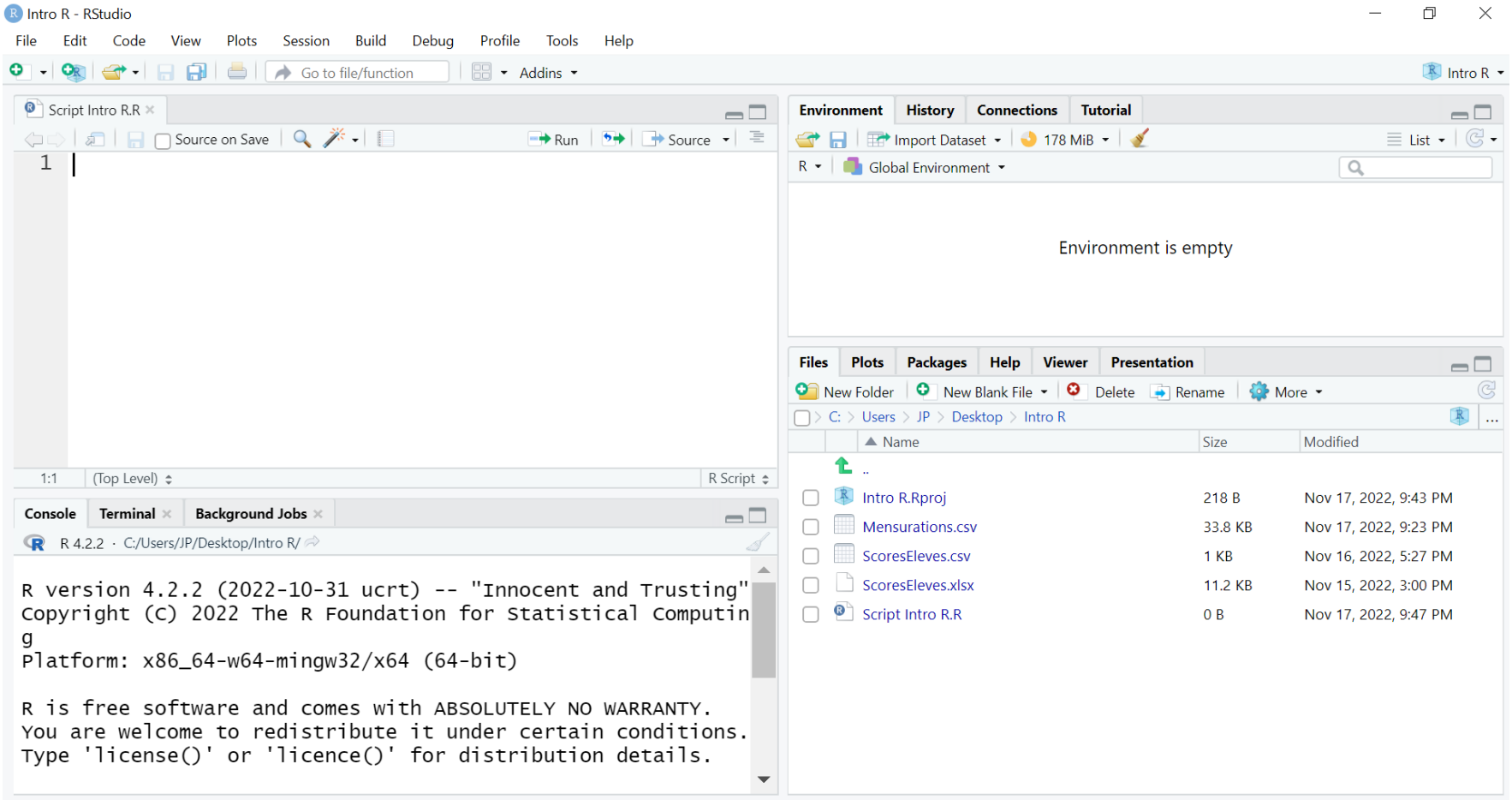


- Sauvegarder le script. L'appeler Script Intro R par exemple. Extension du fichier : .R



Prochaine étape : ouvrir un fichier de script

- Le projet est à présent établi



4. R : Syntaxe

4.1 - R : un langage orienté-objet

Syntaxe

R est un langage *orienté objet*

- Tous les éléments dans R (un chiffre, un tableau de données, un résultat statistique, etc.) peuvent être stockés dans des *boîtes* qu'on appelle **objets**.
- Ce système permet de faire communiquer plus facilement entre eux les éléments manipulés.
- La flèche **`<-`** permet d'**assigner** ou de ranger un élément dans un objet.
- Le code **`a <- 6.35`** crée un objet appelée **`a`** et range le nombre 6.35 à l'intérieur.
- Equivalent : **`a = 6.35`**

R est un langage *orienté objet*

```
a <- 6.35
a
a+7.8
b <- a+7.8
b
Resultat <- sqrt(a) -3*b + a^2
Resultat
Toto <- "I love R"
Toto
```

```
> a <- 6.35
> a
[1] 6.35
> a+7.8
[1] 14.15
> b <- a+7.8
> b
[1] 14.15
> Resultat <- sqrt(a) -3*b + a^2
> Resultat
[1] 0.3924206
> Toto <- "I love R"
> Toto
[1] "I love R"
```

`a, b` et **`Resultat`** sont des nombres. **`Toto`** est une chaîne de caractères.
`sqrt()` permet de calculer la racine carrée.
`a^2` = a exposant 2

R est un langage *orienté objet*

Intro R - RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Script Intro R.R*

```r
a <- 6.35
a
a+7.8
b <- a+7.8
b
Resultat <- sqrt(a) -3*b + a^2
Resultat
Toto <- "I love R"
Toto
```

Console — R 4.2.2 · C:/Users/JP/Desktop/Intro R/

```
> b <- a+7.8
> b
[1] 14.15
> Resultat <- sqrt(a) -3*b + a^2
> Resultat
[1] 0.3924206
> Toto <- "I love R"
> Toto
[1] "I love R"
>
```

Environment — Global Environment

Values	
a	6.35
b	14.15
Resultat	0.392420633670831

Files — C: > Users > JP > Desktop > Intro R

Name	Size	Modified
.Rhistory	0 B	Dec 27, 2022, 4:58 PM
Intro R.Rproj	218 B	Dec 27, 2022, 5:14 PM
Mensurations.csv	33.8 KB	Nov 17, 2022, 9:23 PM
ScoresEleves.csv	1 KB	Nov 16, 2022, 5:27 PM
ScoresEleves.xlsx	11.2 KB	Nov 15, 2022, 3:00 PM
Script Intro R.R	0 B	Nov 18, 2022, 4:20 PM

Liste des objets disponibles dans la session = **environnement**

R est un langage *orienté objet*

- R est **sensible à la casse**. L'objet **`a`** sera différent de l'objet **`A`**. L'objet **`rEsultat`** sera différent de l'objet **`Resultat`**.
- Le nom d'un objet doit commencer par une lettre et peut contenir ensuite des lettres et des chiffres.
- La création d'un objet ayant le même nom qu'un objet existant écrasera l'objet existant.
- La commande **`rm(a)`** efface l'objet **`a`**.

Environnement de travail (Workspace)

- L’ensemble des objets créés lors d’une session peut être sauvegardé sous forme **d’environnement de travail** (fichiers .Rdata).
  - R : Menu file / Save Workspace / Load Workspace.
  - RStudio : Menu Session / Save Workspace / Load Workspace.

4.2 - R : fonctions et packages

Syntaxe

Fonctions : principe

- **Fonctions** : outils permettant de réaliser des **manipulations de données**, des **calculs statistiques** et des **graphiques**.
- Elles s'écrivent de la sorte : **`nomdelafonction(argument 1, argument 2, argument 3…).`**
- Les arguments permettent de paramétrer la fonction.
- Exemple : la fonction **`sqrt()`** écrite **`sqrt(Resultat)`** permet de calculer la racine carrée de l'objet **`Resultat`**.

Fonctions : exemple

- La fonction **`mean`** permet de calculer la moyenne d'une série de données. L'aide de la fonction affiche 3 arguments :
- **`mean(x, trim = 0, na.rm = FALSE)`**
  - Pour faire fonctionner la fonction **`mean`**, L'argument **`x`** est **obligatoire**. Il s'agit de la série de données sur laquelle on cherche à calculer une moyenne.
  - En rouge, valeurs d'arguments par défaut.
  - Les arguments **`trim`** et **`na.rm`** ont une **valeur par défaut** et sont par conséquent **facultatifs**.
  - **`trim`** est le taux de données à chaque extrême à exclure avant le calcul.
  - **`na.rm`** permet d'indiquer si les valeurs manquantes doivent être éliminées (TRUE) ou non (FALSE) avant le calcul.

Exemple d'application de la fonction `mean()`

Je calcule la moyenne de la colonne Pointure issue du jeu de données chaussures...

...En **excluant 10%** des données les plus faibles et 10% les plus élevées...

...Et en **excluant** les données manquantes

```
mean(x = chaussures$Pointure, trim = 0.1, na.rm = TRUE)
```

Fonctions : aide

- Arguments des fonctions expliqués dans l'aide de la fonction. Pour accéder à l'aide d'une fonction :

  `?mean` (ou F1 sur la fonction dans RStudio)

- L'aide comprend différentes sections :
  - Le nom de la fonction et le package associé : **`fonction{package}`**
  - Une description brève
  - Les arguments et leur explication
  - Les résultats fournis par la fonction (Value)
  - Les références bibliographiques
  - Un exemple de script exécutable directement sous R

mean {base} R Docum

Arithmetic Mean

Description

Generic function for the (trimmed) arithmetic mean.

Usage

```
mean(x, ...)

## Default S3 method:
mean(x, trim = 0, na.rm = FALSE, ...)
```

Arguments

x	An R object. Currently there are methods for numeric/logical vectors and date-time and time interval objects. Complex vectors are allowed for tr 0, only

Exemple d'application de la fonction `mean()`

Je calcule la moyenne de la colonne Pointure issue du jeu de données chaussures...

...En **excluant 10%** des données les plus faibles et 10% les plus élevées...

...Et en **excluant** les données manquantes

```
mean(x = chaussures$Pointure, trim = 0.1, na.rm = TRUE)
```

- Raccourci (non recommandé) :

```
mean(chaussures$Pointure, 0.1, TRUE)
```

packages

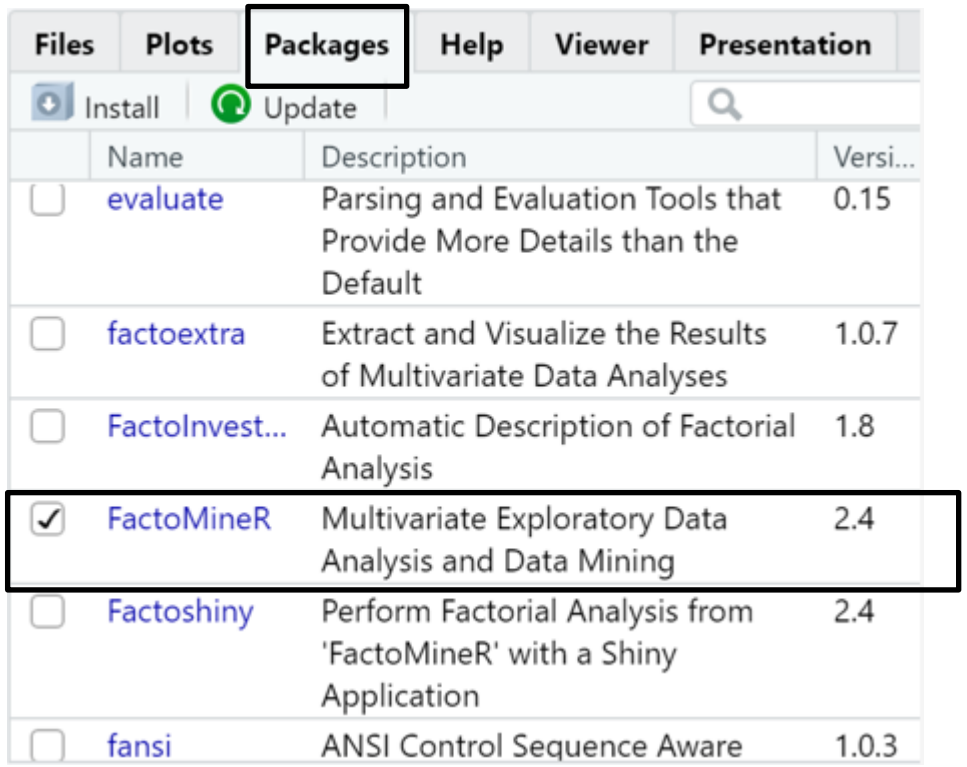
- Les fonctions sont groupées en ***packages*** (= bibliothèques = librairies)
- Une fois installé, le logiciel R comprend des packages de base.
- La liste des packages installés se trouve dans la fenêtre en bas à droite de RStudio (cliquer sur Packages)
- Pour installer un nouveau package (exemple : FactoMineR) :
  - RStudio : **menu Tools / Install packages**
  - Ou taper : `install.packages("FactoMineR")`

Le package FactoMineR est utilisé pour les statistiques exploratoires multivariées

packages

- Pour avoir accès aux fonctions d'un package, il faut charger le package. Ceci est indispensable d'une session à une autre
- Dans la fenêtre en bas à droite, *chochez* les packages que vous souhaitez charger.
- Sinon, utiliser le code :

```
library(FactoMineR)
```



Quelques packages courants

- **`{dplyr}`** et **`{tidyr}`** : Manipulations de tableaux de données (Tidyverse).
- **`{stringr}`** : Manipulations de chaînes de caractère (Tidyverse).
- **`{ggplot2}`** : Librairie graphique, fonctionne avec des calques (Tidyverse).
- **`{plotly}`** : Librairie pour graphiques interactifs.
- **`{caret}`** : Machine Learning supervisé.
- **`{FactoMineR}`** et **`{factoextra}`** : Statistiques exploratoires multivariées.
- **`{cluster}`** : Clustering (segmentation).

Récapitulatif : fonctionnement d’un projet R avec RStudio

Répertoire courant (projet)

c:/JP/mes projets/projet R formation

- données1.csv
- données2.csv
- sorties1.csv
- Script1.R
- Script2.R
- Env1.Rdata

...

R

environnement de travail

objet1
objet2
objet3
...

Calculs

package1
fonction1
fonction2
fonction3

package2
fonction4
fonction5

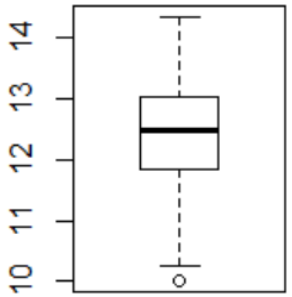
Script1.R

```
library(package1)
objet1 <- read.csv("données.csv")
boxplot(objet1)
objet2 <- fonction1(objet1)
objet2
```

Console

```
> boxplot(objet1)
> objet2 <- fonction1(ob
> objet2
[1] 12.43594
>
```

Graphique



RStudio (partie *visible*)

4.3 - R : Principaux types de données et commentaires

Syntaxe

Les types de données

- Principaux types :
  - **Numérique** *numeric* (englobe positif, négatif, entier, réel)
  - **Chaîne de caractère** *character :* `"I love R"`
  - **Logique** *logical* (vrai/faux : T/F, TRUE/FALSE)
  - **Valeur manquante** = NA
  - **Date** : donnée de date: `as.Date("2023-02-24")`
- Type d'un objet accessible via la fonction `class()`

```
Jesuisunchiffre <- 5
class(Jesuisunchiffre)
[1] "numeric"
```

#Commentaires

- Pour ajouter des commentaires au script, utiliser #

```
sqrt(5) #Calculer la racine carrée de 5

#Multiplier a par    l'exponentielle de 10 et stocker
le    résultat dans l'objet a
a<-exp(10)*a
```

Tout ce qui se trouve après # ne sera pas interprété par R

4.4 – Syntaxe classique et Tidyverse

Syntaxe

Syntaxe classique et Tidyverse

- Depuis la naissance du langage R, la syntaxe a beaucoup évolué.
- Elle devient de plus en plus ergonomique.
- Meilleure fluidité depuis l'apparition du Tidyverse dont voici quelques packages principaux:
  - **`dplyr`**: opérations sur les tableaux de données
  - **`tidyr`**: remodelage de tableaux de données
  - **`lubridate`**: manipulation de données temporelles
  - **`stringr`**: opérations sur chaînes de caractères
  - **`readr`** et **`readxl`**: import/export de jeux de données
  - **`purrr`**: opérations sur listes
  - **`ggplot2`**: construction de graphiques
  - ...
- Dans cette formation nous alternerons entre syntaxe classique et syntaxe Tidyverse.

5. Les data frames (tableaux de données)

Quelques mots sur les vecteurs, avant tout

Les vecteurs

- **Vecteurs** (*vectors*) : Structure à une dimension. Suite d'éléments ordonnés et de même type. Un vecteur peut représenter une variable. Différentes fonctions permettent de créer des vecteurs :
  - `c()` #combiner des éléments
  - `seq()` #créer une séquence de chiffres
  - `rep()` #répéter des éléments
- Mesurer la taille d'un vecteur (nombre d'éléments) :
  - `length(vecteur)`

Vecteurs, création

```
#Combiner des éléments
vec1 <- c(2, 4.3, 6, 4.5) #Combiner des éléments
vec1
[1] 2.0 4.3 6.0 4.5

#Appliquer une opération sur chaque élément du vecteur
vec2<-vec1*2 + 3
vec2
[1] 7.0 11.6 15.0 12.0

#Additionner deux vecteurs, élément par élément (fonctionne aussi pour les opérations
- ; / ; * et ^)
vec1 + vec2
[1] 9.0 15.9 21.0 16.5

vec3 <- 3:14
vec3
[1]  3  4  5  6  7  8  9 10 11 12 13 14
```

Vecteurs, création

```
# Concaténer deux vecteurs en un vecteur + grand
# la fonction sqrt() permet de calculer la racine carrée
vec4 <- c(sqrt(vec2), c(4,9.8))
vec4
[1] 2.645751 3.405877 3.872983 3.464102 4.000000 9.800000

# Créer un vecteur de chaînes de caractères
Regions<- c("Region1", "Region2", "Region3")

#Une autre manière de procéder : la fonction paste concatène
des chaînes de caractère
Regions <- paste("Region", c(1,2,3), sep="_")
Regions
[1] "Region_1" "Region_2" "Region_3"
```

chaîne de caractère séparant les chaînes à coller

Passons à présent aux jeux de données

Jeu de données illustratif

Le jeu de ScoresEleves **est inventé** et correspond à des scores d'examens obtenus par 30 élèves d'origines, de genres et d'établissements différents. Disponible aux formats .xslx et .csv

Elèves

ID	Origine	Genre	Etablissement	Score1	Score2	Score3	Score4	Score5
N1	O3	M	A	51.1	29.1	7.1	6.2	25
N2	O2	M	D	74.5	100	87.1	87.7	79.3
N3	O3	M	A	53.2	51.9	12.9	9.6	0
N4	O3	M	C	76.6	13.6	15.7	15.8	30
N5	O2	F	B	14.9	27.1	100	100	92.9
N6	O3	M	C	85.1	21.1	18.6	17.5	13.6
N7	O3	F	C	80.9	54.2	22.9	21.9	17.1
N8	O3	F	A	100	55	25.7	25.4	23.6
N9	O1	M	A	29.8	34.5	2.9	1.8	40
N10	O1	F	C	76.6	52.9	4.3	2.6	3.6

Data frames (tableaux de données)

- **Data frame**: Structure à deux dimensions. La majorité des jeux de données classiques (variables en colonnes, individus ou observations en lignes) sont importés sous forme de *data frame* dans R.
- Les possibilités de **manipulations** de data frame sont infinies. Dans cette formation, nous nous focaliserons sur l'essentiel, avec des fonctions issues de packages de base ou du package **`{dplyr}`**.

Travailler avec des data frames

1. Préparer un jeu de données et l'Importer en data frame
2. Caractériser un data frame
3. Accéder à différentes parties d'un data frame (`[ , ]`, `select`, `filter`)
4. Trier un data frame (`arrange`)
5. Créer de nouvelles colonnes (`mutate`)
6. Effectuer automatiquement un calcul sur chaque colonne
7. Effectuer des calculs par groupe
8. Grouper les opérations : utilisation du *pipe*

5.1 – Préparer et importer des fichiers (.txt, .csv…) sous forme de data.frame

La structure data.frame

Format courant de fichier de données : .csv

- Il s'agit d'un format "texte" où les colonnes sont séparées par un caractère spécifique (virgule ou point virgule).
- Il est possible de l'ouvrir sous Excel.
- Il est possible de sauvegarder un fichier Excel au format .csv (sauvegarder sous, et sélectionner l'extension .csv dans la liste)

ScoresEleves - Notepad

File Edit Format View Help

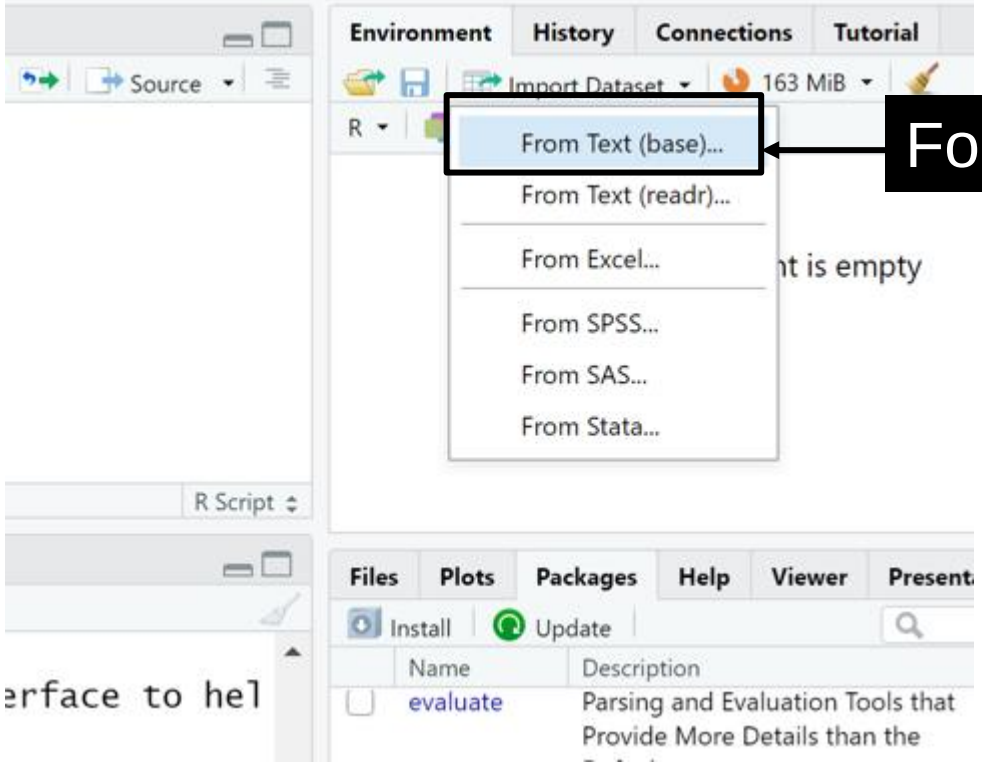
```
ID;Origine;Genre;Etablissement;Score1;Score2;Score3;Score4;Score5
N1;O3;M;A;51,1;29,1;7,1;6,2;25
N2;O2;M;D;74,5;100;87,1;87,7;79,3
N3;O3;M;A;53,2;51,9;12,9;9,6;0
N4;O3;M;C;76,6;13,6;15,7;15,8;30
N5;O2;F;B;14,9;27,1;100;100;92,9
N6;O3;M;C;85,1;21,1;18,6;17,5;13,6
N7;O3;F;C;80,9;54,2;22,9;21,9;17,1
N8;O3;F;A;100;55;25,7;25,4;23,6
N9;O1;M;A;29,8;34,5;2,9;1,8;40
N10;O1;F;C;76,6;52,9;4,3;2,6;3,6
N11;O2;F;C;23,4;62,2;88,6;92,1;99,3
N12;O3;F;A;40,4;54,7;31,4;30,7;21,4
N13;O1;F;C;42,6;22,8;5,7;3,5;2,9
N14;O3;M;B;59,6;19,7;28,6;27,2;34,3
N15;O1;F;B;95,8;50,5;4,3;1,8;15,7
```

Avant d’importer son tableau : nettoyer son fichier

- Excel : ranger les données à partir de la cellule A1.
- Un tableau unique dans le fichier.
- Observations ou individus en lignes et des variables en colonnes.
- Inclure les noms des variables sur une ligne unique (la première).
- Eviter les cellules fusionnées.
- Eviter les caractères spéciaux comme / \ ; < > - + : , |
- Privilégier les caractères *ne posant pas de problèmes* : _ et .
- Eviter les noms de variables trop longs.
- Utiliser une chaîne de caractère unique pour les données manquantes (ou laisser vides les cellules concernées)

Importer un jeu de données .csv ou .txt dans R avec R Studio

Pour importer un jeu de données .csv ou .txt dans R via RStudio, cliquer sur *Import Dataset*, puis From text et se laisser guider...



Format .csv ou .txt

Importer un jeu de données .csv ou .txt dans R avec R Studio

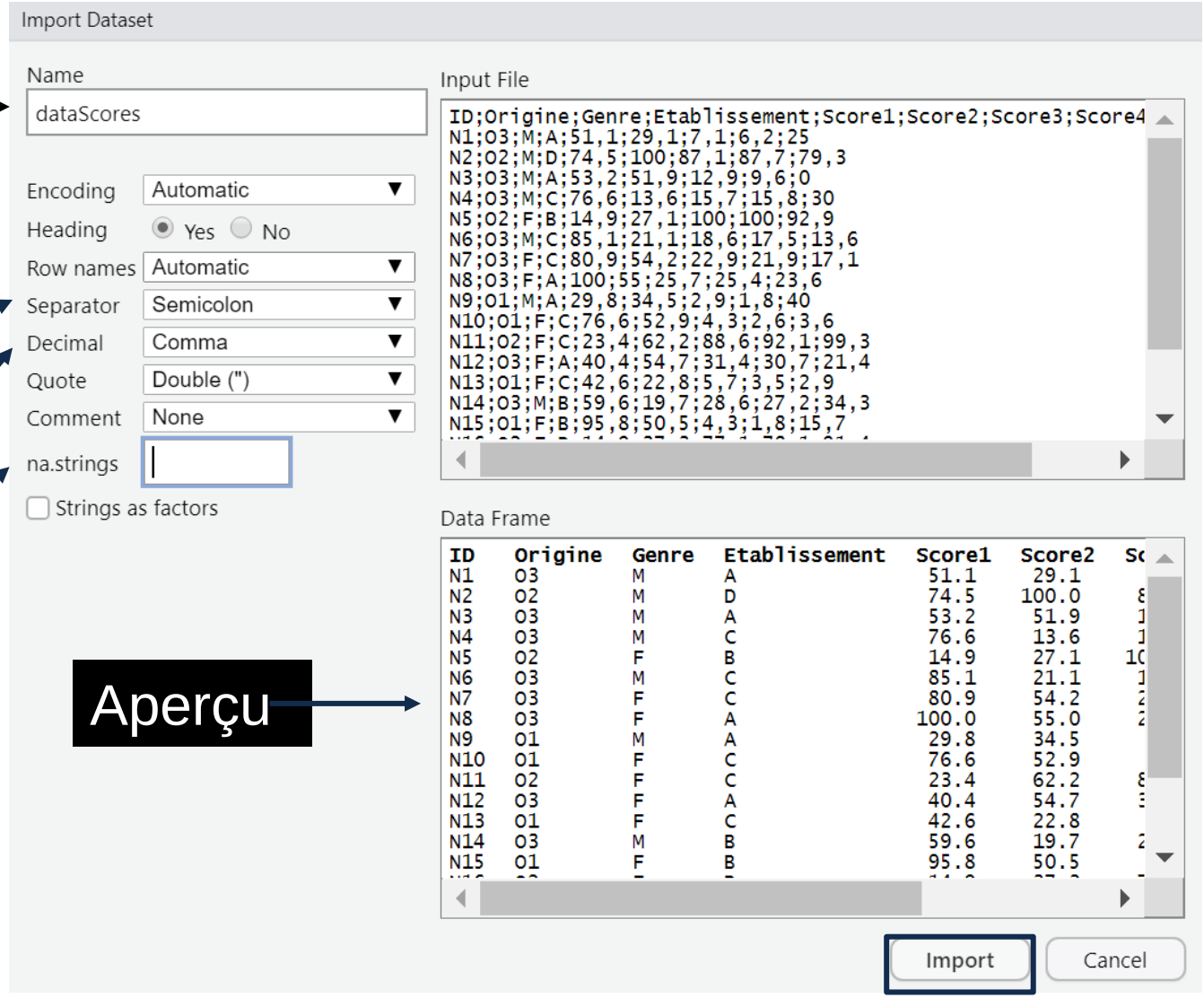
nom de l'objet R dans lequel sera stocké le jeu importé

La première ligne correspond-elle aux noms des variables ?

Séparateur de champs (colonnes)

Séparateur décimal

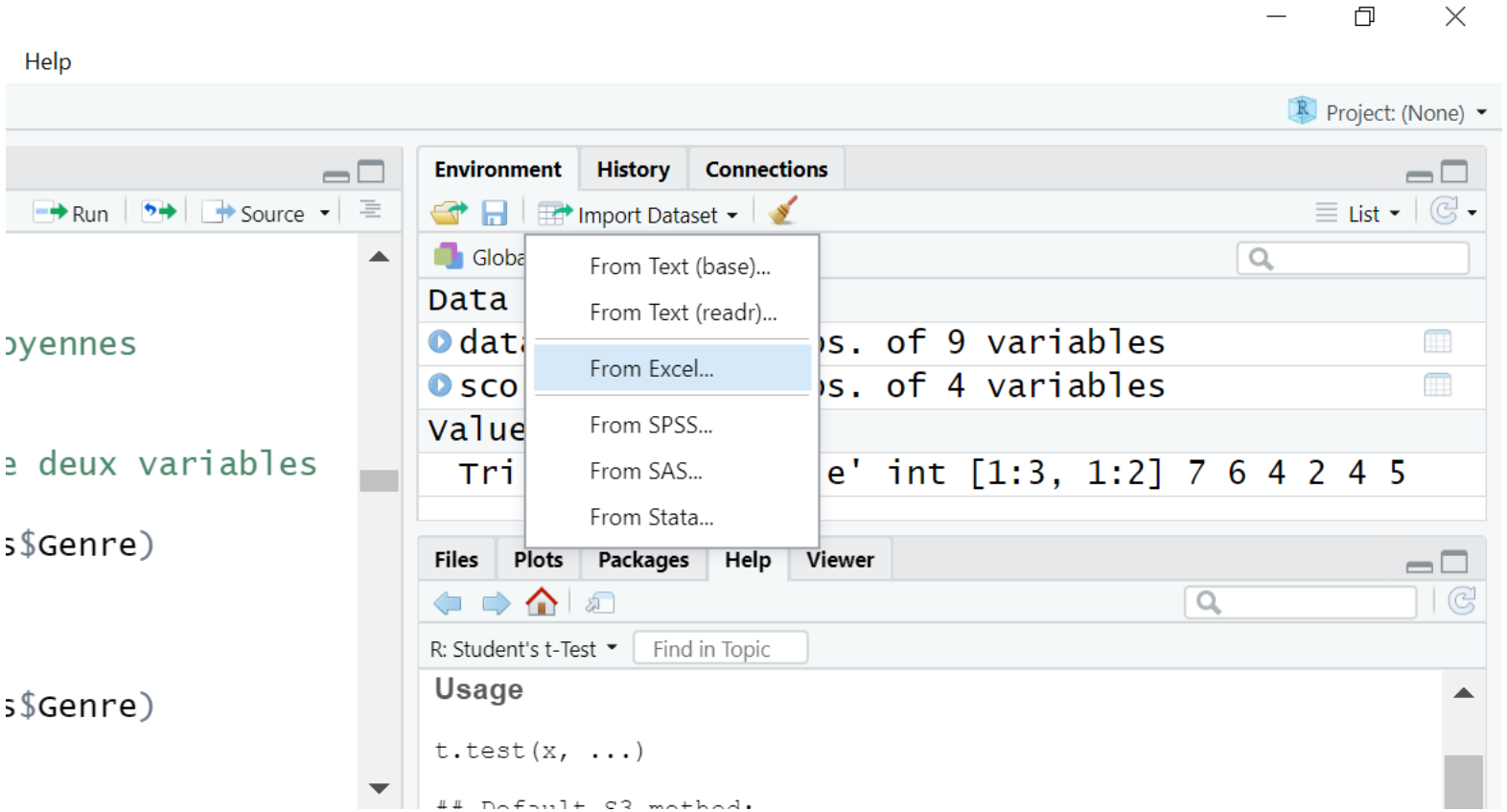
Indicateur de données manquantes (effacer le contenu si les données manquantes correspondent à des cellules vides)



Aperçu

Importer un jeu de données Excel dans R avec R Studio

Pour importer un jeu de données Excel dans R via R Studio, cliquer sur *Import Dataset*, et se laisser guider...



Importer un jeu de données Excel dans R avec R Studio

Aperçu

Import Excel Data

File/URL:

C:/Users/JP/Desktop/190919_UniversiteBordeaux/ScoresEleves.xlsx

Browse...

Data Preview:

ID (character)	Origine (character)	Genre (character)	Etablissement (character)	Score1 (double)	Score2 (double)	Score3 (double)	Score4 (double)	Score5 (double)
N1	O3	M	A	51.1	29.1	7.1	6.2	25.0
N2	O2	M	D	74.5	100.0	87.1	87.7	79.
N3	O3	M	A	53.2	51.9	12.9	9.6	0.0
N4	O3	M	C	76.6	13.6	15.7	15.8	30.
N5	O2	F	B	14.9	27.1	100.0	100.0	92.9
N6	O3	M	C	85.1	21.1	18.6	17.5	13.6

Previewing first 50 entries.

Rechercher le fichier à importer

Import Options:

Name: dataScores

Sheet: Default

Range: A1:D10

Max Rows:

Skip: 0

NA:

First Row as Names

Open Data Viewer

Code Preview:

```
library(readxl)
dataScores <- read_excel("ScoresEleves
.xlsx")
View(dataScores)
```

Import

Cancel

nom de l'objet R dans lequel sera stocké le jeu importé

La première ligne correspond-elle aux noms des variables ?

Autres fonctions d'importation

```
# librairie readr pour lire des fichiers .txt ou .csv
library(readr)
dataScores <- read_csv2(file="ScoresEleves.csv")
Rows: 30 Columns: 9
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (4): ID, Origine, Genre, Etablissement
dbl (5): Score1, Score2, Score3, Score4, Score5
[...]

# Fonction fread issue de la librairie data.table pour lire différents formats
library(data.table)
dataScores<-fread("ScoresEleves.csv", dec=",")

# librairie readxl pour lire des fichiers Excel
library(readxl)
dataScores<-read_excel("ScoresEleves.xlsx")
```

Remplacer read_csv2 par read_csv si les champs sont séparés par “,”

Fonctions *plus classiques* d'importation (.csv ou .txt)

```
dataScores<-read.table(file="ScoresEleves.csv", header=T,
                       sep=",", dec=".", na.strings="", stringsAsFactors=T)
```

Arguments :

- **`file =`** Chemin du fichier (ou URL). Si vous mettez le nom du fichier seul, assurez-vous d'être dans le bon répertoire courant.
- **`header =`** présence d'une ligne d'en-tête avec les noms des colonnes à créer
- **`sep =`** caractère séparateur
  (`"\t"` = tabulation ; `","` ou `";"` = fichiers csv en général)
- **`dec =`** séparateur décimal (entre guillemets)
- **`nrows =`** nombre maxi de lignes à importer
- **`na.sgrings =`** indicateur de valeurs manquantes dans le fichier. S'il s'agit de cellules vides, indiquer **`na.strings=""`**.
- **`stringsAsFactors =`** importer les colonnes de chaînes de caractères en facteurs ?

!! Peuvent changer selon les paramètres régionaux

Attention : se mettre dans le bon répertoire courant, sinon spécifier le chemin précis dans **`read.table`**

5.2 - Caractériser un data.frame

La structure data.frame

Caractériser un data.frame

```
# Dimensions (nb. de lignes, nb. de colonnes)
dim(dataScores)
[1] 30  9

#Nombre de colonnes
ncol(dataScores)
[1] 9

#Nombre de lignes
nrow(dataScores)
[1] 30

#Noms des colonnes (vecteur)
colnames(dataScores)
[1] "ID"            "Origine"       "Genre"
[4] "Etablissement" "Score1"        "Score2"
[7] "Score3"        "Score4"        "Score5"
```

Caractériser un data.frame

```
# Quelques premières lignes
head(dataScores)
  ID Origine Genre Etablissement Score1 Score2 Score3 Score4 Score5
1 N1      O3     M             A   51.1   29.1    7.1    6.2   25.0
2 N2      O2     M             D   74.5  100.0   87.1   87.7   79.3
3 N3      O3     M             A   53.2   51.9   12.9    9.6    0.0
4 N4      O3     M             C   76.6   13.6   15.7   15.8   30.0
5 N5      O2     F             B   14.9   27.1  100.0  100.0   92.9
6 N6      O3     M             C   85.1   21.1   18.6   17.5   13.6

# Quelques dernières lignes
tail(dataScores)
```

5.3 - Accéder à différentes parties du data.frame

La structure data.frame

Accéder à différentes parties du data.frame

- 3 objectifs différents :
  - Accéder à des lignes / colonnes suivant leurs **numéros** → `[ , ]`.
  - Accéder à des **colonnes** particulières suivant leurs **intitulés** → fonction **`select{dplyr}`**
  - Extraire un sous-ensemble de lignes suivant certaines conditions. Exemple : toutes les observations pour lesquelles Score4 < 60 → fonction **`filter{dplyr}`**

Accès aux éléments du tableau suivant les numéros des lignes / colonnes

data.frame - accès aux éléments du tableau suivant les numéros des lignes / colonnes

- **Accès aux éléments du tableau** suivant les numéros des lignes et des colonnes

```
tableau[ , ]
```

Sélectionner numéros de lignes

Sélectionner numéros de colonnes

data.frame : accès aux éléments du tableau suivant les indices

```
# ligne 2, colonne 3
dataScores[2,3]
[1] "M"

# lignes 1 à 4, colonne 2
dataScores[1:4,2]
[1] "O3" "O2" "O3" "O3"

# lignes 2 et 5, toutes les colonnes
dataScores[c(2,5),]
  ID Origine Genre Etablissement Score1 Score2 Score3 Score4 Score5
2 N2      O2     M             D   74.5  100.0   87.1   87.7   79.3
5 N5      O2     F             B   14.9   27.1  100.0  100.0   92.9
```

data.frame : accès aux éléments du tableau suivant les indices

```
# lignes 1 et 3, colonnes 3 et 2
dataScores[c(1,3),c(3,2)]
  Genre Origine
1     M      O3
3     M      O3

# accès à la colonne "Score5"
dataScores$Score5
 [1]  25.0  79.3   0.0  30.0  92.9  13.6  17.1  23.6  40.0   3.6  99.3
[12]  21.4   2.9  34.3  15.7  91.4  52.9  20.0  68.6  79.3  25.0  10.0
[23]  41.4 100.0  95.7  13.6  17.1  24.3  28.6  70.7

# calculer la moyenne de Score5
mean(dataScores$Score5)
[1] 41.24333
```

Sélectionner des colonnes suivant les intitulés : `select{dplyr}`

Sélectionner des colonnes avec `select()`

- On cherche à effectuer des sélections d'une ou plusieurs colonnes en utilisant les noms. Utilisation de la fonction **`select()`** issue de **`{dplyr}`**

```
library(dplyr)

# Garder uniquement Origine et Score5
select(dataScores, Origine, Score5)

# Prendre tout sauf Origine et Score5
select(dataScores, -Origine, -Score5)

# Prendre toutes les colonnes dont le nom commence avec "Score"
select(dataScores, starts_with("Score"))

# Exclure toutes les colonnes dont le nom finit avec "ement"
select(dataScores, -ends_with("ement"))
```

Sélectionner des colonnes avec `select()`

```
# Exclure toutes les colonnes dont le nom contient "en"
select(dataScores, -contains("en"))

# Exclure toutes les colonnes dont le nom contient un
"t" au milieu
select(dataScores, -matches(".t."))

# Sélectionner toutes les colonnes entre Origine et
Score1
select(dataScores, Origine:Score1)
```

Sélectionner des lignes suivant des conditions : `filter{dplyr}`

Filtrer les lignes avec `filter()`

- On cherche à effectuer des sélections de lignes particulières dans le jeu de données, selon des **filtres** appliqués sur les colonnes.
- Exemple : toutes les lignes pour lesquelles Score4 > 40 et Score5 < 80.
- Utilisation de la fonction **`filter()`** issue de **`{dplyr}`**

Opérateur de filtrage	Signification
`<`	inférieur à
`>`	supérieur à
`<=`	inférieur ou égal à
`>=`	supérieur ou égal à
`==`	égal à
`!=`	différent de
`&`	et
`|`	ou

Filtrer les lignes avec `filter()`

```
library(dplyr)

# Conserver uniquement les lignes avec Score4 > 90
filter(dataScores, Score4 > 90)
   ID Origine Genre Etablissement Score1 Score2 Score3 Score4 Score5
1  N5      O2     F             B   14.9   27.1  100.0  100.0   92.9
2 N11      O2     F             C   23.4   62.2   88.6   92.1   99.3
3 N24      O2     M             C    0.0   31.6   97.1  100.0  100.0
4 N25      O2     F             C   53.2   47.7   87.1   91.2   95.7
5 N30      O2     F             A   36.2   58.0   94.3   95.6   70.7
```

Filtrer les lignes avec `filter()`

```
# Filtrer Origine O1 et Score1 > 50
filter(dataScores, Origine == "O1", Score1 > 50)

# Filtrer Origines 1 ou 3
filter(dataScores, Origine == "O1" | Origine =="O3")

# Exclure l'établissement C
filter(dataScores, Etablissement != "C")
```

5.4 – Trier un data frame

La structure data.frame

Trier avec `arrange()`

- On cherche à trier le tableau de données par ordre alpha-numérique selon des colonnes ciblées. Utilisation de la fonction **`arrange()`** issue de **`{dplyr}`**

```
# trier en fonction de la colonne Score1, ordre croissant
arrange(dataScores, Score1)
    ID Origine Genre Etablissement Score1 Score2 Score3 Score4 Score5
1  N24      O2     M             C    0.0   31.6   97.1  100.0  100.0
2   N5      O2     F             B   14.9   27.1  100.0  100.0   92.9
3  N16      O2     F             B   14.9   37.3   77.1   78.1   91.4
4  N19      O2     F             D   21.3   13.2   82.9   86.0   68.6
5  N11      O2     F             C   23.4   62.2   88.6   92.1   99.3
...

# ...ordre décroissant
arrange(dataScores, -Score1)
```

Trier avec `arrange()`

```
# trier en par ordre d'établissement, puis de Score3 décroissant
arrange(dataScores, Etablissement, -Score3)
    ID Origine Genre Etablissement Score1 Score2 Score3 Score4 Score5
1  N30      O2     F             A   36.2   58.0   94.3   95.6   70.7
2  N20      O2     M             A   23.4   43.2   87.1   86.8   79.3
3  N12      O3     F             A   40.4   54.7   31.4   30.7   21.4
4   N8      O3     F             A  100.0   55.0   25.7   25.4   23.6
5   N3      O3     M             A   53.2   51.9   12.9    9.6    0.0
6   N1      O3     M             A   51.1   29.1    7.1    6.2   25.0
7  N29      O1     F             A   91.5   53.3    4.3   50.0   28.6
8   N9      O1     M             A   29.8   34.5    2.9    1.8   40.0
9  N26      O1     M             A   70.2   34.5    2.9    0.9   13.6
10 N18      O1     F             A   83.0   31.2    1.4    0.9   20.0
11  N5      O2     F             B   14.9   27.1  100.0  100.0   92.9
12 N16      O2     F             B   14.9   37.3   77.1   78.1   91.4
13 N14      O3     M             B   59.6   19.7   28.6   27.2   34.3
...
```

5.5 – Créer de nouvelles colonnes

La structure data.frame

Créer de nouvelles colonnes avec `mutate()`

- La création de nouvelles colonnes peut se faire facilement avec la fonction **`mutate()`** issue de **`{dplyr}`**

```
# créer une colonne Score5_sur_10 correspondant aux Scores5
divisés par 10
mutate(dataScores, Score5_sur_10 = Score5/10)

# créer une colonne MoyS1S2 correspondant à la moyenne des
Scores 1 et 2
mutate(dataScores, MoyS1S2 = (Score1+Score2)/2)
```

Créer de nouvelles colonnes avec `mutate()`

```
# grouper les deux dernières commandes, sauvegarder dans un nouvel objet
# et afficher les 7 dernières colonnes de l'objet
dataScores_new<- mutate(dataScores, Score5_sur_10 = Score5/10,
                                    MoyS1S2 = (Score1+Score2)/2)
dataScores_new[,5:11]
 Score1 Score2 Score3 Score4 Score5 Score5_sur_10 MoyS1S2
1   51.1   29.1    7.1    6.2   25.0          2.50   40.10
2   74.5  100.0   87.1   87.7   79.3          7.93   87.25
3   53.2   51.9   12.9    9.6    0.0          0.00   52.55
4   76.6   13.6   15.7   15.8   30.0          3.00   45.10
5   14.9   27.1  100.0  100.0   92.9          9.29   21.00
6   85.1   21.1   18.6   17.5   13.6          1.36   53.10
7   80.9   54.2   22.9   21.9   17.1          1.71   67.55
8  100.0   55.0   25.7   25.4   23.6          2.36   77.50
9   29.8   34.5    2.9    1.8   40.0          4.00   32.15
...
```

5.6 – Effectuer un calcul sur chaque colonne automatiquement

La structure data.frame

Effectuer un calcul automatiquement sur chaque colonne avec `apply()`

Le 2 signifie *exécuter la fonction par colonne*. Pour exécuter par ligne, remplacer par 1.

- **`apply(data.frame, MARGIN = 2, fonction)`** permet d'exécuter une fonction sur chaque colonne d'un data.frame.

```
# Calculer la moyenne de chaque colonne de dataScores[,5:9]
apply(dataScores[,5:9], MARGIN = 2,mean)
 Score1    Score2    Score3    Score4    Score5
53.91333 40.35000 36.71333 36.40333 41.24333

# Calculer la moyenne de chaque ligne de dataScores[,5:9]
apply(dataScores[,5:9], MARGIN = 1,mean)
 [1] 23.70 85.72 25.52 30.34 66.98 31.18 39.40 45.94 21.80 28.00 73.12
[12] 35.72 15.50 33.88 33.62 59.76 49.82 27.30 54.40 63.96 30.94 33.64
[23] 15.98 65.74 74.98 24.42 33.72 19.80 35.90 70.96
```

Effectuer un calcul automatiquement sur chaque colonne avec `summarise_all()`

- Pour gagner du temps, il est possible d'exécuter un calcul automatiquement sur chaque colonne d'un jeu de données. Utilisation de la fonction **`summarise_all()`** de la librairie **`{dplyr}`**

```
# Calculer la moyenne de chaque colonne
summarise_all(dataScores, mean)
  ID Origine Genre Etablissement   Score1 Score2   Score3 Score4   Score5
1 NA      NA    NA            NA 53.91333  40.35 36.71333  38.01 41.24333
# Un message de warning apparaît, parce que la moyenne ne peut être
calculée sur des colonnes non-numériques
```

Effectuer un calcul automatiquement sur chaque colonne avec `summarise_all()`

```
# En relançant le calcul sur les colonnes numériques (numéros 5 à 9) :
summarise_all(dataScores[,5:9], mean)
    Score1 Score2   Score3 Score4   Score5
1 53.91333  40.35 36.71333  38.01 41.24333

# Equivalent avec summarise_if() :
summarise_if(dataScores, is.numeric, mean)
# lecture : sélectionner les colonnes numériques dans dataScores, et
calculer la moyenne sur chaque colonne
```

5.7 – Effectuer des calculs par groupe

La structure data.frame

Effectuer des calculs par groupe avec `group_by()` et `summarise()`

- L'automatisation du calcul peut aussi se faire par groupes dans le jeu de données. Par exemple, on peut s'intéresser à la moyenne d'un certain score en fonction de chaque origine. Nous utiliserons les fonctions **`group_by()`** et **`summarise()`** issues du package **`{dplyr}`**

```
# On crée un objet Ori décomposant le jeu dataScores par Origine
Ori <- group_by(dataScores, Origine)

# On utilise cet objet pour calculer des variables agrégées par Origine :
Score1 moyen et somme de Score2
summarise(Ori, Score1_moyen = mean(Score1),
           Score2_somme = sum(Score2))
Origine   Score1_moyen Score2_somme
1 O1              66.2         364.
2 O2              28.7         456.
3 O3              66.8         391.
```

5.8 – Grouper les opérations : utilisation du *pipe*

La structure data.frame

dplyr : utilisation du pipe %>%

- Le *pipe* %>% permet d'enchaîner rapidement les opérations **dplyr** sur un jeu de données.

```
dataScores%>%
  filter(Etablissement == "A" | Etablissement == "C")%>%
  arrange(Origine, -Score5)%>%
  mutate(Moy35 = (Score3+Score5)/2)%>%
  select(-ID, Genre, -Score1, -Score2)
```

- Explications :
  - On prend le tableau dataScores
  - On conserve uniquement les lignes correspondant aux établissements A et C
  - On trie par ordre alphabétique d'origine puis par ordre décroissant de Score5
  - On crée une colonne Moy35 correspondant à la moyenne des scores 3 et 5
  - On exclue les colonnes ID, Genre, Score1 et Score2

6. Graphiques avec R

Graphiques de base avec R

- Les packages de base offrent de **nombreuses fonctions** graphiques permettant une personnalisation très précise. Ces fonctions sont faciles à manier s'il y a peu de personnalisation : **`plot()`**, **`boxplot()`**, **`histogram()`**, **`barplot()`**...
- Chaque package peut proposer des fonctions graphiques spécialisées.
- La fonction **`plot_ly()`** issue du package **`{plotly}`** permet de générer des graphiques interactifs.
- La fonction **`ggplot()`** issue du package **`{ggplot2}`** propose une syntaxe particulière permettant de construire rapidement des graphiques complexes avec peu de code.
- La majorité des graphiques de cette formation seront produits avec **`ggplot()`**

Graphiques avec `ggplot()`

Graphiques avec R

La fonction `ggplot{ggplot2}`

- La fonction **`ggplot{ggplot2}`** permet de construire des graphiques simples ou complexes.
- On commence par construire la **base** du graphique avec la fonction **`ggplot()`**, qui comporte 2 arguments :
  - **`data`** : indiquer le data frame renfermant les données en entrée.
  - **`mapping`** : indiquer les variables qui se placeront sur x, sur y, qui détermineront les couleurs, etc.
- A cette base, on superpose un ou plusieurs *calques esthétiques* (*layers*).
- Chaque calque est une ligne de code. Les calques sont séparés par des **+** (jamais en début de ligne).

`ggplot()` : Construire la **base** du graphique

```
ggplot(data, mapping = aes(x, y, color, fill, shape, label))
```

- `data` : data frame sur lequel travailler
- `x` : Variable à placer sur l'axe x
- `y` : Variable à placer sur l'axe y
- `color` : Variable de coloration (points, bordures...)
- `fill` : Variable déterminant la couleur de remplissage
- `shape` : Variable déterminant le type de figuré (points)
- `label` : Variable contenant le contenu d'étiquettes (texte)

Tous ces arguments sont *optionnels*

`ggplot()` : Exemple de base de graphique

```
ggplot(data=dataScores, mapping=aes(x=Score1, y=Score5, color=Origine))
```

ID	Origine	Genre	Etablissement	Score1	Score2	Score3	Score4	Score5
N1	O3	M	A	51.1	29.1	7.1	6.2	25.0
N2	O2	M	D	74.5	100.0	87.1	87.7	79.3
N3	O3	M	A	53.2	51.9	12.9	9.6	0.0
N4	O3	M	C	76.6	13.6	15.7	15.8	30.0
N5	O2	F	B	14.9	27.1	100.0	100.0	92.9

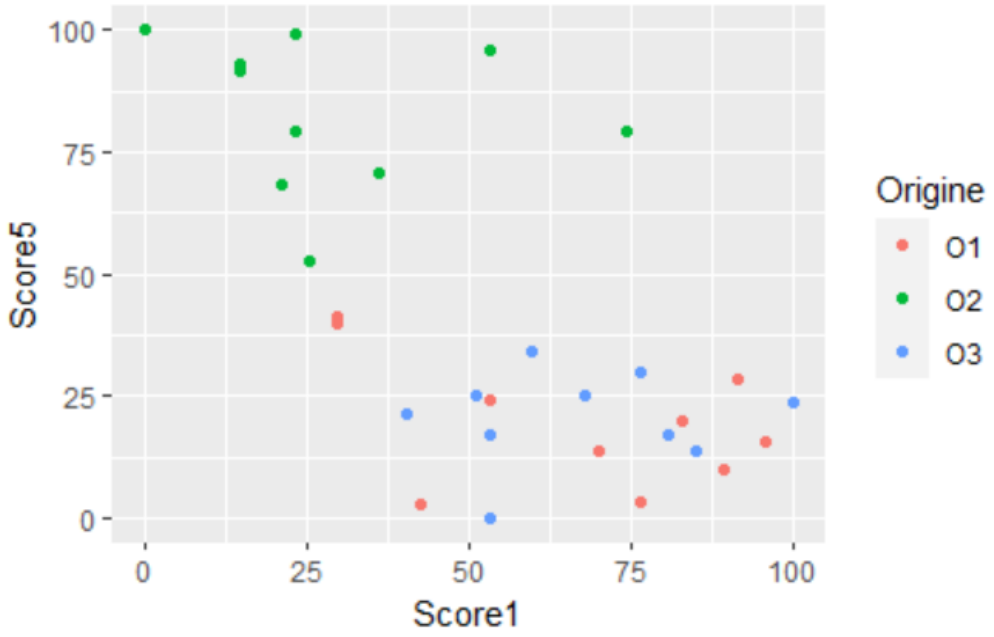
- Cette commande produit un graphique vide.
- Il faut ajouter des calques : figurés ? titres ? thèmes ?...

ggplot() : On rajoute un calque simple

```
ggplot(data=dataScores, mapping=aes(x=Score1, y=Score5, color=Origine))+
      geom_point()
```

Calque “points”

Pour rajouter un calque au graphique, mettre un + en fin de ligne



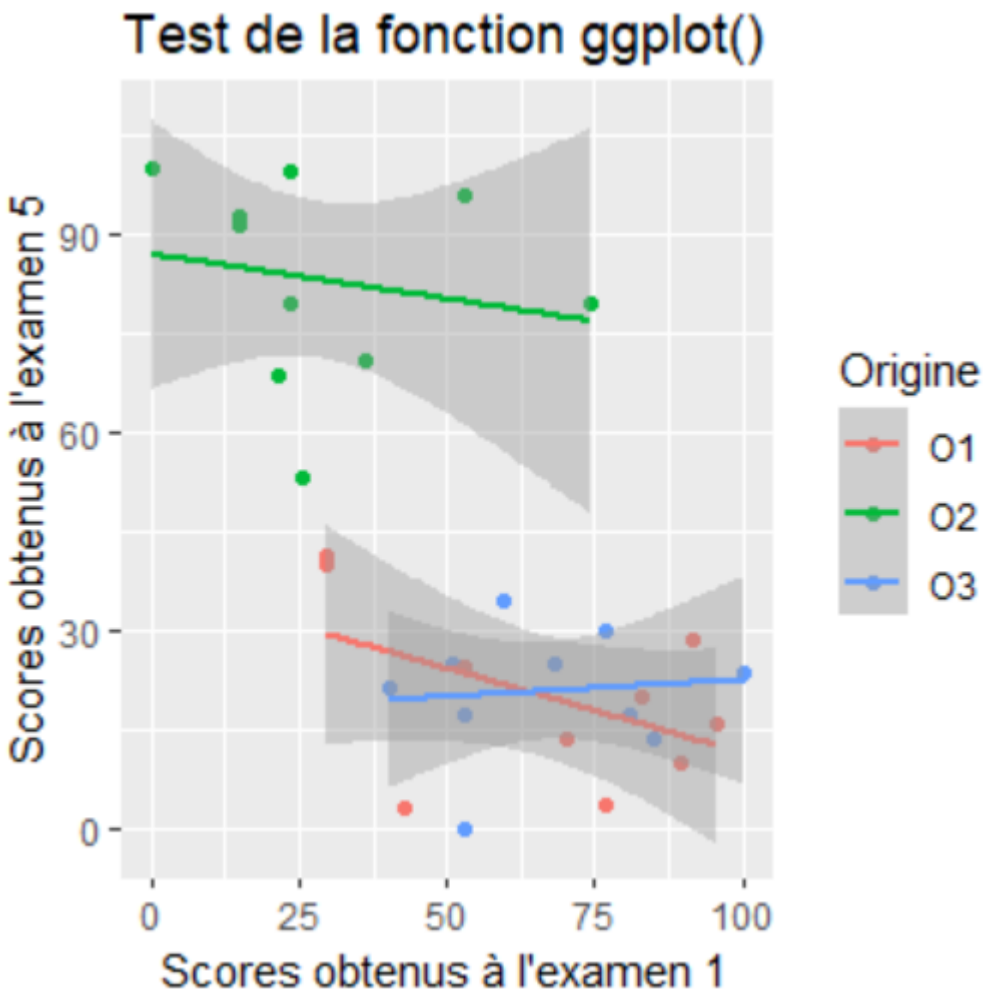
Calques courants

Calque	Définition	Exemple
`geom_bar()`	Diagramme en bâtons	
`geom_histogram()`	Histogramme	
`geom_density()`	Densité	
`geom_point()`	Nuage de points	
`geom_text()`	Ajouter des étiquettes	
`geom_smooth()`	Ajouter des courbes de tendance	`geom_smooth(method="lm")`
`geom_boxplot()`	Box plot	
`facet_wrap()`	Subdiviser la fenêtre graphique selon une variable groupe	`facet_wrap(facets=~Origine)`
`scale_color_manual()`	Changer la couleur des figurés manuellement (points...)	`scale_color_manual(values=c("red", "darkgreen", "#00008B"))`
`scale_fill_manual()`	Changer la couleur de remplissage manuellement (histogramme, boxplot...)	`scale_fill_manual(values=c("red", "darkgreen", "#00008B"))`
`ggtitle()`	Configurer le titre du graphique	`ggtitle("Ceci est un titre")`
`xlab()`	Configurer le titre de l'axe x	`xlab("Mon titre pour l'axe des x")`
`ylab()`	Configurer le titre de l'axe y	`ylab("Mon titre pour l'axe des y")`
`theme_bw()`, `theme_classic()`, `theme_dark()`, `theme_minimal()...`	Utiliser un thème particulier de graphique	

ggplot() : Enchaînement de calques, exemple 1

```
ggplot(data=dataScores, mapping=aes(x=Score1,
y=Score5, color=Origine))+
        geom_point()+
        geom_smooth(method="lm")+
        ggtitle("Test de la fonction ggplot()")+
        ylab("Scores obtenus à l'examen 5")+
        xlab("Scores obtenus à l'examen 1")+
        theme_bw()
```

Utiliser le thème graphique **bw** (fond blanc, cadre noir...)

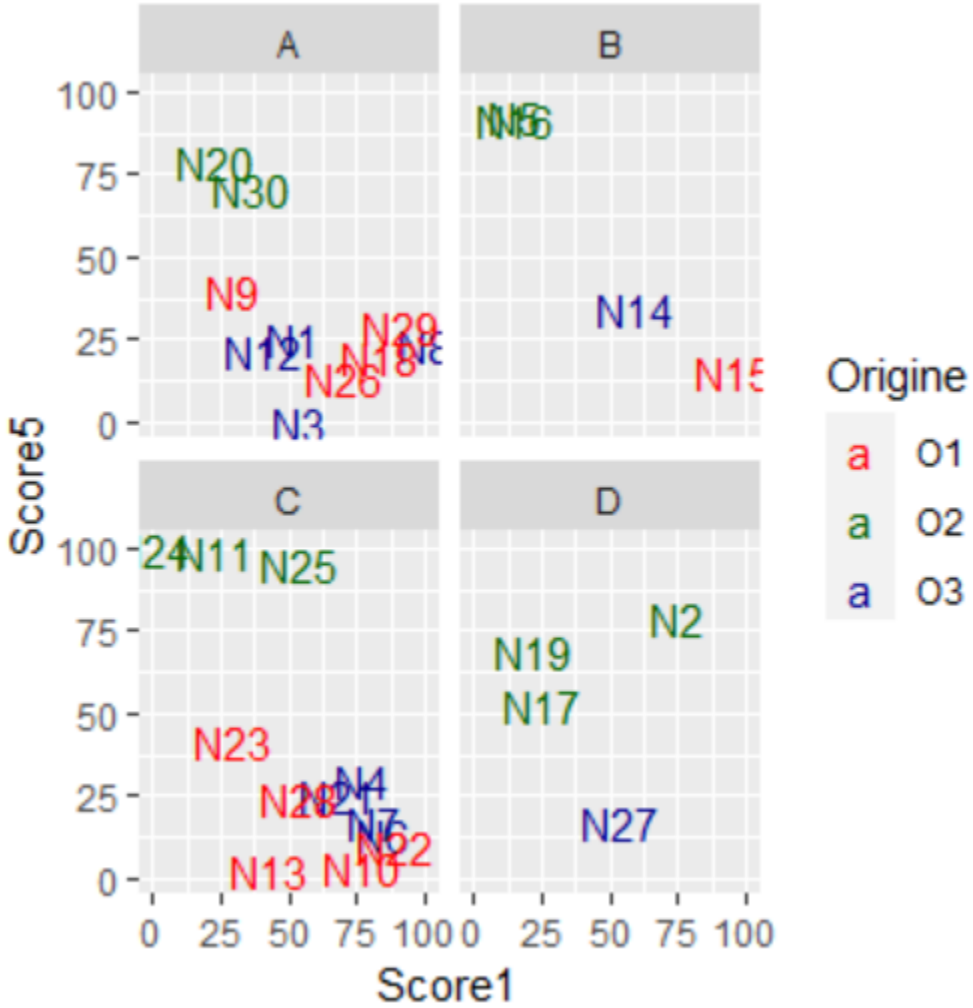


ggplot() : Enchaînement de calques, exemple 2

```
ggplot(data=dataScores, mapping=aes(x=Score1,
y=Score5, color=Origine, label=ID))+
        geom_text()+
        facet_wrap(facets=~Etablissement)+
        scale_color_manual(values=c("red",
                "darkgreen", "#00008B"))
```

Il y aura une fenêtre par établissement

Changer les couleurs manuellement. Combiner les couleurs avec la fonction `c()`



Fin !

Des questions ?

Vous pouvez me contacter sur
www.statistique-formation.fr